

Post-Quantum Cryptography

PKI Test Lab

Standing up quantum-safe certificate authorities end to end — issuing ML-DSA certificates and negotiating ML-KEM hybrid key exchange over TLS 1.3.

PART 1

Windows Server 2025 + AD CS

Enterprise PKI with ML-DSA certificates, IIS, and Group Policy client enablement.

PART 2

Ubuntu 26.04 LTS + Apache

Root and issuing CA via LibOQS or Microsoft SymCrypt, with hybrid TLS on Apache.

ML-DSA

ML-KEM

TLS 1.3

AD CS

Apache

OpenSSL

James Ringold

Director, Security Solution Engineering · Microsoft · Greater Pittsburgh Region

25+

Years in Security

CISO

Former, Westinghouse

CISSP

CCSP · MBA

ISSA

Pittsburgh President



Background

25+ years across retail, aerospace, defense, and nuclear energy. Former CISO at Westinghouse Electric Company, security architect, and incident responder who has rebuilt major security programs after incidents.



Today at Microsoft

Leads the Security Solution Engineering team, helping customers across industries get real value from Defender, Sentinel, Purview, Entra, and Security Copilot — tying security investment to business outcomes.



Credentials

CISSP (ISC)² since 2004 and CCSP, plus Microsoft security certifications across Azure and Microsoft 365. MBA from the University of Minnesota's Carlson School of Management.



Speaking & Community

President of ISSA Pittsburgh and an organizer behind Three Rivers Information Security Symposium – Career Village. Speaks on quantum-safe cryptography, cloud security, and threat trends; writes for the Microsoft Security Blog.

Algorithm Primer

ML-DSA (FIPS 204 · Dilithium)

Role	Digital Signature Algorithm
Used for	Certificate and CA key signing
In certificates?	YES — the certificate KEY algorithm
ml-dsa-65	NIST Level 3 · Server & Intermediate CA
ml-dsa-87	NIST Level 5 · Root CA (highest security)

ML-KEM (FIPS 203 · Kyber)

Role	Key Encapsulation Mechanism
Used for	TLS 1.3 key exchange ONLY
In certificates?	NO — handshake-level group
x25519_mlkem768	Hybrid · Recommended for internet
mlkem1024	Pure PQ · Controlled networks

⚠ ML-KEM is a Key Encapsulation Mechanism — it is NEVER a certificate key algorithm.

Building a Post-Quantum Cryptography PKI Test Lab

Windows Server 2025 GA (build 26100.33158+, KB5099536) • or vNext Build 29550+

ML-DSA

ML-KEM

AD CS

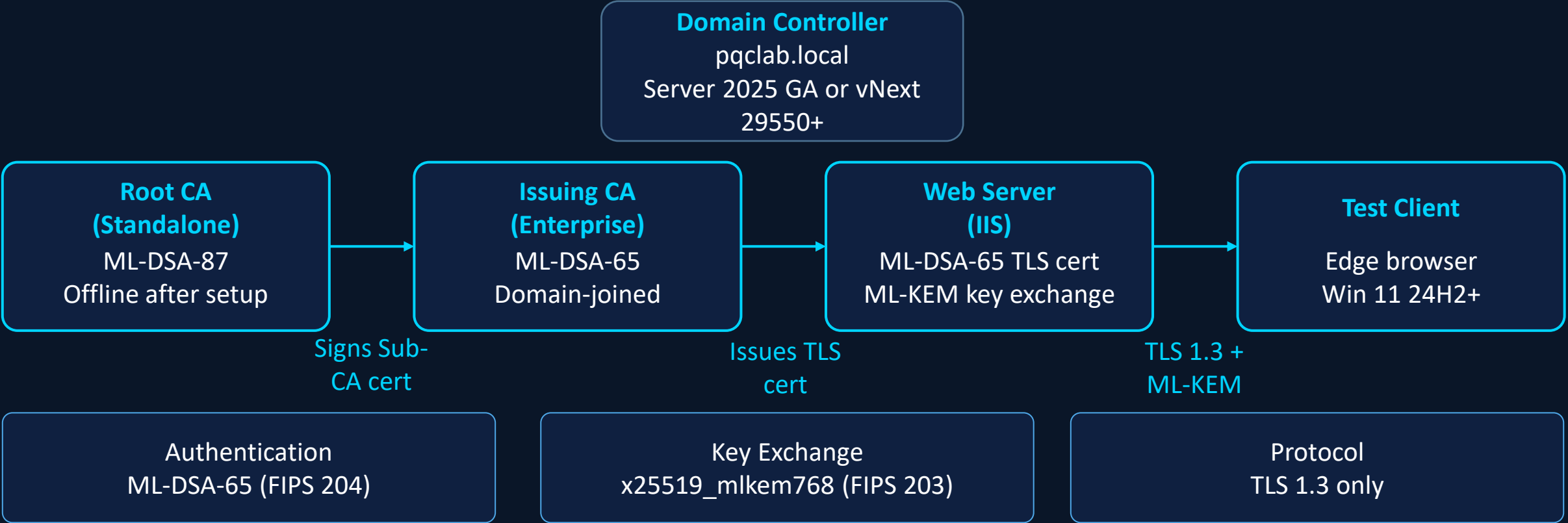
TLS 1.3

IIS

Known Limitations

Limitation	Impact
ML-DSA is signature-only	Cert auth uses ML-DSA; key exchange uses ML-KEM separately in the TLS 1.3 handshake — two independent PQC mechanisms.
No Composite certs yet (Phase 2)	Cannot combine classical + ML-DSA in a single certificate. Pure ML-DSA only for now.
No ML-KEM public keys in certs	ML-KEM operates only in TLS 1.3 handshake key exchange — not embedded in the certificate itself.
ML-KEM groups disabled by default	Must be explicitly enabled on both server AND client via Group Policy or TLS cmdlets.
TLS 1.3 only for ML-KEM	Hybrid groups (x25519_mlkem768, etc.) are NOT available in TLS 1.2 or below.
Client compatibility	ML-DSA cert validation requires Win 11 24H2/25H2 + KB5067036+. Chrome/Firefox unconfirmed — use Edge for testing.
Insider build instability	Build 29550+ is pre-release. NOT for production. Test lab only. The GA versions were released in July 2026. Do not support hybrid certificates, yet.

Lab Architecture



Minimum 2 VMs (combine Issuing CA + DC), ideally 3-4 VMs.
All VMs must run Windows Server 2025 GA (build 26100.33158+ with KB5099536) or vNext build 29550+.
Standalone Root CA can be offline (non-domain-joined) after signing the Sub-CA certificate.

Phase 1 — Choose the Server Build

Obtain Windows Server 2025 GA ISO (or vNext Preview) and deploy your lab VMs

Setup Steps

1. Choose the build path
GA: Windows Server 2025 + KB5099536
OR vNext: Windows Server Insider Preview
2. Download Windows Server 2025 GA ISO
(build 26100.33158+ with KB5099536) OR
Windows Server vNext Preview Build 29550+
3. Deploy on VMs — Standard or Datacenter
edition both supported
4. No additional KBs required
Build 29550+ includes ML-DSA and ML-KEM natively

Lab VM Layout

VM 1 — Root CA

Standalone • Offline after setup
Server 2025 GA + KB5099536 or vNext 29550+

VM 2 — Domain Controller

Can combine with Issuing CA
Server 2025 GA + KB5099536 or vNext 29550+

VM 3 — Issuing CA

Enterprise • Domain-joined
Server 2025 GA + KB5099536 or vNext 29550+

VM 4 — Web Server

IIS • Domain-joined
Server 2025 GA + KB5099536 or vNext 29550+

Phase 2 — Active Directory Setup

Required for Enterprise CA • Skip if using Standalone-only lab

1. Install AD DS Role

```
Install-WindowsFeature AD-Domain-Services -  
IncludeManagementTools
```

2. Promote to Domain Controller

```
Install-ADDSForest `  
-DomainName "pqclab.local" `  
-DomainNetbiosName "PQCLAB" `  
-InstallDns `  
-Force
```

3. Join other VMs to the domain

```
Add-Computer -DomainName pqclab.local -Restart
```

Run step 3 on the Issuing CA and Web Server VMs.

Why Use an Enterprise CA?

- **Auto-enrollment**
Computers and users receive certificates automatically via Group Policy — no manual requests needed.
- **Template management**
Publish and version certificate templates through Active Directory; changes deploy instantly.
- **CRL distribution**
Certificate revocation lists and OCSP responses are published to AD automatically.
- **Trust propagation**
Root CA certificate is distributed to all domain members via GPO — no manual trust steps on clients.

Phase 3 — Root CA Setup (ML-DSA-87)

On the Root CA VM • Can be Standalone (non-domain-joined)

1. Install AD CS Role

```
Install-WindowsFeature ADCS-Cert-Authority -
IncludeManagementTools
```

2. Configure Root CA with ML-DSA-87

```
# KeyLength = 2592 bytes x 8 = 20736 bits
Install-AdcsCertificationAuthority `
  -CAType StandaloneRootCA `
  -CACommonName "PQCLab Root CA" `
  -KeyLength 20736 `
  -HashAlgorithm NoHash `
  -CryptoProviderName "ML-DSA:87#Microsoft Software Key
Storage Provider" `
  -ValidityPeriod Years -ValidityPeriodUnits 10 -Force
```

3. Verify the Root CA Certificate

```
certutil -store Root "PQCLab Root CA"
```

ML-DSA Parameter Sets

Set	Pub Key	Sig Size	NIST Lvl
ML-DSA-44	1,312 B	2,420 B	Level 2
ML-DSA-65	1,952 B	3,309 B	Level 3
ML-DSA-87 *	2,592 B	4,627 B	Level 5

After setup: take Root CA offline

After the Root CA signs the Subordinate CA certificate, shut the Root CA VM down and keep it offline. This is PKI best practice — the Root CA private key must never be exposed on the network.

Phase 4 — Issuing (Subordinate) CA Setup

On the Issuing CA VM • Must be domain-joined • ML-DSA-65

1. Install & Configure Subordinate CA

```
Install-WindowsFeature ADCS-Cert-Authority -  
IncludeManagementTools
```

```
# ML-DSA-65: 1952 bytes x 8 = 15616 bits
```

```
Install-AdcsCertificationAuthority `
```

```
-CAType EnterpriseSubordinateCA `
```

```
-CACommonName "PQCLab Issuing CA" `
```

```
-KeyLength 15616 `
```

```
-HashAlgorithm NoHash `
```

```
-CryptoProviderName "ML-DSA:65#Microsoft Software Key  
Storage Provider"
```

2. Submit CSR to Root CA + Install Cert

```
# On Root CA: submit the saved .req file
```

```
certutil -submit SubCA.req
```

```
# Approve in certsrv.msc > Pending Requests > Issue
```

```
# On Issuing CA: accept the signed certificate
```

```
certreq -accept SubCA.cer
```

```
# Publish Root CA cert so domain clients trust it
```

```
certutil -dspublish -f RootCA.cer RootCA
```

**Install
AD CS Role**

**Configure
Sub-CA
(ML-DSA-65)**

**Save
.req File**

**Submit to
Root CA**

**Accept
.cer Cert**

Verify: In certsrv.msc > CA Properties > View Certificate > Details tab: Signature Algorithm = ML-DSA-87 (signed by Root), Public Key = ML-DSA-65 (Sub-CA own key).

Phase 5 — Create PQC TLS Certificate Template

In certtmpl.msc • Duplicate the built-in Web Server template

Compatibility CA: Windows Server 2008 or later Recipient: Windows Vista / Server 2008 or later	General Display name: PQC Web Server Validity period: 1 year	Request Handling (CRITICAL) Change Purpose to: Signature ML-DSA is a signing algorithm only — this MUST be changed from the default.
Cryptography Provider Category: Key Storage Provider Algorithm name: ML-DSA:65	Extensions Application Policies: Server Authentication only Do NOT include EFS or Secure Email EKUs.	Security Grant Enroll permission to the web server computer account or Domain Computers group.
# Publish the template on the Issuing CA Add-CATemplate -Name 'PQC Web Server'		

Phase 6 — Enroll TLS Certificate on Web Server

On the Web Server VM • Install IIS first • Use local computer cert store

Method A — GUI (certlm.msc)

- 1 Install IIS: Server Manager > Add Roles > Web Server (IIS)
- 2 Open certlm.msc (Local Computer certificate store)
- 3 Personal > Certificates > All Tasks > Request New Certificate
- 4 Select the PQC Web Server template > Enroll
- 5 IIS Manager > Site > Bindings > Add HTTPS > select ML-DSA cert

Method B — certreq (scripted)

```
; request.inf
[NewRequest]
Subject = "CN=webserver.pqclab.local"
KeyUsage = 0x80
KeySpec = AT_SIGNATURE
ProviderName = "Microsoft Software Key Storage Provider"
HashAlgorithm = NoHash
KeyAlgorithm = "ML-DSA:65"
RequestType = PKCS10

[EnhancedKeyUsageExtension]
OID=1.3.6.1.5.5.7.3.1 ; Server Authentication

[RequestAttributes]
CertificateTemplate = PQC Web Server
```

```
certreq -new request.inf request.req
certreq -submit -config "IssuingCA\PQCLab Issuing CA" request.req
cert.cer
certreq -accept cert.cer
```

Phase 7 — Enable ML-KEM Hybrid TLS Key Exchange

Run on BOTH the web server AND test client • Disabled by default in build 29550+

Method A — PowerShell

```
# Enable ML-KEM hybrid key exchange groups
Enable-TlsEccCurve -Name "x25519_mlkem768" # Recommended
Enable-TlsEccCurve -Name "secp256r1_mlkem768"
Enable-TlsEccCurve -Name "secp384r1_mlkem1024"

# Set group priority (PQC groups first)
$groups = "x25519_mlkem768","secp256r1_mlkem768",
          "secp384r1_mlkem1024","curve25519","nistP256","nistP384"
Set-TlsEccCurve -Name $groups
```

Method B — Group Policy

Computer Config > Admin Templates > Network > SSL Config
Settings > ECC Curve Order

```
x25519_mlkem768,secp256r1_mlkem768,secp384r1_mlkem1024,curve25519,nistP256,nistP384
```

Supported Hybrid Groups (TLS 1.3 only)

Group	Classical	PQC (FIPS 203)
x25519_mlkem768 *	X25519	ML-KEM-768
secp256r1_mlkem768	NIST P-256	ML-KEM-768
secp384r1_mlkem1024	NIST P-384	ML-KEM-1024

TLS 1.3 Handshake with PQC

- ClientHello → offers x25519_mlkem768
- ServerHello → selects x25519_mlkem768
- Server sends ML-DSA-65 certificate
- Hybrid key exchange → shared secret

Phase 8 — Trust Distribution to Test Clients

Distribute Root CA cert so clients trust the ML-DSA certificate chain

Method A — Domain GPO (auto-distributes to all domain members)

```
# For Standalone Root CA, publish to AD manually:  
certutil -dspublish -f RootCA.cer RootCA  
  
# For Enterprise Root CA, cert auto-publishes.  
# All domain members receive it via Group Policy automatically.
```

Method B — Manual import (non-domain or workgroup clients)

```
# Export Root CA cert from the Root CA server  
certutil -store Root "PQCLab Root CA" rootca.cer  
  
# On each test client (run as Administrator):  
Import-Certificate -FilePath .\rootca.cer -CertStoreLocation Cert:\LocalMachine\Root
```

Browser Compatibility for ML-DSA Certificate Validation

Microsoft Edge

Supported

Uses Windows CNG — validates ML-DSA chains natively.

Chrome

Unknown

Uses BoringSSL — ML-DSA validation not confirmed as of July 2026.

Firefox

Unknown

Uses NSS — ML-DSA validation not confirmed as of July 2026.

curl / .NET 10

Supported

Uses Windows CNG on Windows. .NET 10 has native MLKemCng support.

Phase 9 — Verify Everything

Confirm PQC authentication and key exchange are working end-to-end

PowerShell TLS Connection Test

```
$tcp = [System.Net.Sockets.TcpClient]::new("webserver.pqclab.local",  
443)  
$ssl = [System.Net.Security.SslStream]::new($tcp.GetStream())  
$ssl.AuthenticateAsClient("webserver.pqclab.local")  
Write-Host "Protocol: $($ssl.SslProtocol)"  
Write-Host "Key Exchange: $($ssl.KeyExchangeAlgorithm)"  
Write-Host "Cipher: $($ssl.CipherAlgorithm)"
```

certutil — Verify Certificate

```
certutil -verify -urlfetch webserver.pqclab.local.cer
```

```
Get-ChildItem Cert:\LocalMachine\My |  
Where-Object {$_.Subject -like "*webserver*"} |  
Format-List Subject, Thumbprint, NotAfter
```

Wireshark Validation

```
# Display filter: tls.handshake  
# In ClientHello look for Supported Groups:  
# 0x11ec = x25519_mlkem768 <- PQC active  
# 0x11eb = secp256r1_mlkem768  
# 0x11ed = secp384r1_mlkem1024
```

Success Checklist

- Root CA cert: ML-DSA-87 signature and public key
- Issuing CA cert: ML-DSA-87 (signed by Root), ML-DSA-65 (own key)
- Web server cert: ML-DSA-65 public key, Server Auth EKU
- TLS protocol shown as TLS 1.3
- Key exchange group name contains 'mlkem'
- Edge browser shows valid HTTPS, no cert warnings

Phase 10 — Windows 11 Client Verification

Win11 24H2 GA + KB5101650 • or Insider Preview 26100.8514+ • Enable ML-KEM • Confirm PQC TLS in Edge DevTools

1. Enable ML-KEM on Win11 Client

July 14, 2026 Update: GA Win11 24H2 + KB5101650 (build 26100.8524+) is now sufficient.
Win11 26H1 + KB5095091 also works. Insider Preview 26100.8514+ still valid.
Server side: Server 2025 GA + KB5099536 now supports ML-KEM TLS (same cmdlets).
BOTH endpoints must advertise ML-KEM groups in ClientHello for negotiation to succeed.

```
Enable-TlsEccCurve -Name "x25519_mlkem768"  
Enable-TlsEccCurve -Name "secp256r1_mlkem768"  
Enable-TlsEccCurve -Name "secp384r1_mlkem1024"
```

```
Set-TlsEccCurve -Name @(  
    "x25519_mlkem768",    # ML-KEM-768 + X25519 (preferred)  
    "secp256r1_mlkem768", # ML-KEM-768 + P-256  
    "secp384r1_mlkem1024", # ML-KEM-1024 + P-384  
    "NistP384","NistP256","x25519" # Classical fallbacks  
)
```

```
# Ensure TLS 1.3 Client is enabled (ML-KEM is TLS 1.3-only)  
$k = "HKLM:\SYSTEM\CurrentControlSet\Control\SecurityProviders" +  
    "\SCHANNEL\Protocols\TLS 1.3\Client"  
if (-not (Test-Path $k)) { New-Item $k -Force | Out-Null }  
Set-ItemProperty $k -Name Enabled -Value 1 -Type DWord
```

```
gpupdate /force # Pull Root CA trust from domain GPO  
Restart-Computer -Force
```

2. Verify in Microsoft Edge (F12 DevTools)

- 1 Open Edge, browse to <https://webserver01.pqclab.local>
- 2 Padlock → Connection is secure → Certificate is valid
- 3 Cert details: Signature Algorithm = id-ML-DSA-65
- 4 Cert chain: Issuer = PQCLab Issuing CA (ML-DSA-87 Root)
- 5 F12 → Security tab → Connection section
- 6 Key exchange: x25519_mlkem768 • Protocol: TLS 1.3

Expected PQC TLS Profile

Protocol: TLS 1.3 • Cipher: AES-256-GCM

Key exchange: x25519_mlkem768 (FIPS 203) • Cert
signature: id-ML-DSA-65 (FIPS 204)

What You End Up With

Certificate Chain

PQCLab Root CA

ML-DSA-87 • Self-signed • 10yr • Offline



PQCLab Issuing CA

ML-DSA-65 • Signed by Root • Enterprise CA



webserver.pqclab.local

ML-DSA-65 • Server Authentication • 1yr

TLS 1.3 Connection Profile

Authentication:	ML-DSA-65 certificate (FIPS 204)
Key Exchange:	x25519_mlkem768 hybrid (FIPS 203)
Session Cipher:	AES-256-GCM (TLS 1.3 standard)
Quantum threat:	Harvest-now-decrypt-later mitigated

Phase 2 Roadmap (Planned)

- Composite ML-DSA: classical + PQ in one certificate
- Composite ML-KEM: hybrid key encapsulation in certs
- ML-KEM public keys embedded in certificates
- Full CEP, CES, NDES, OCSP support for PQC

Post-Quantum Certificate Authority

Ubuntu 26.04 LTS · ML-DSA · ML-KEM · Apache TLS

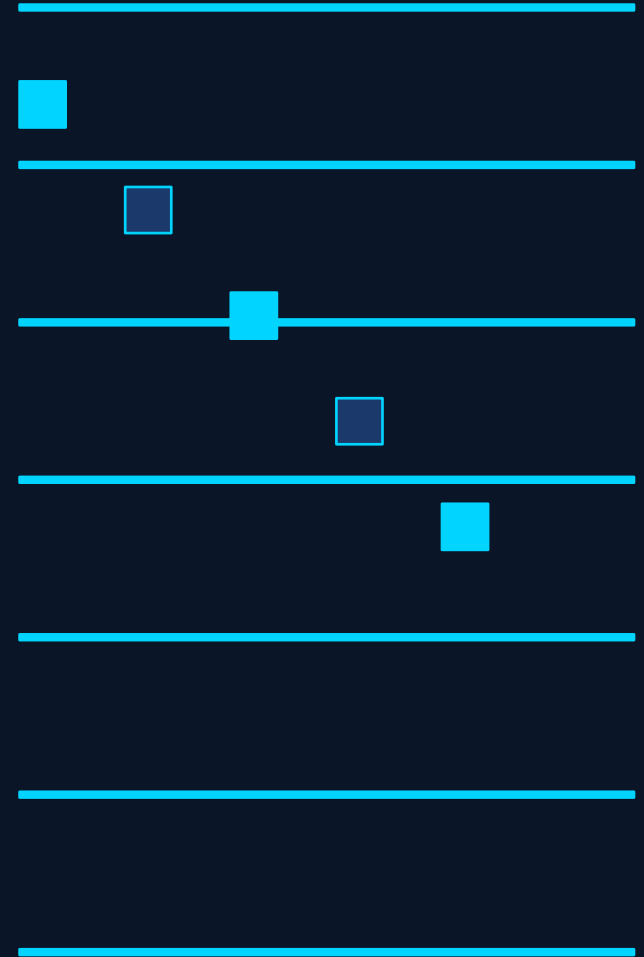
Option A: LibOQS + oqs-provider

Option B: Microsoft SymCrypt

FIPS 203

FIPS 204

NIST PQC



Architecture Overview

ML-DSA applies to:

- Root CA private key
- Intermediate CA key
- Server certificate key
- All cert signatures

ml-dsa-87 → Root

ml-dsa-65 → Intermediate

ml-dsa-65 → Server cert

Root CA

ML-DSA-87 · Self-signed · 20 yr · Air-gapped

Intermediate CA

ML-DSA-65 · Signed by Root · 10 yr

Server Certificate

ML-DSA-65 · Signed by Intermediate · 825 days

Apache 2.4 / mod_ssl

TLS 1.3 · x25519_mlkem768 key exchange

ML-KEM applies to:

- TLS 1.3 handshake
- Apache Groups directive

x25519_mlkem768

Hybrid (recommended)

mlkem1024

Pure PQ

System Preparation

Ubuntu 26.04 LTS · OpenSSL 3.x required for provider architecture

1 Install Dependencies

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y \
  build-essential cmake ninja-build git \
  libssl-dev python3-pip python3-pytest \
  apache2 libapache2-mod-ssl \
  astyle doxygen graphviz unzip
```

2 Verify OpenSSL 3.x

```
openssl version
# Must show: OpenSSL 3.x.x
# Ubuntu 26.04 ships OpenSSL 3.3+
# Provider architecture requires 3.x
```

3 Create Directory Structure

```
export CA_ROOT=/opt/pq-ca
sudo mkdir -p $CA_ROOT/{root-ca,intermediate-ca}
sudo mkdir -p $CA_ROOT/root-ca/{certs,crl,
  newcerts,private,csr}
sudo chmod 700 $CA_ROOT/root-ca/private
```

4 Initialize CA Database

```
sudo touch $CA_ROOT/root-ca/index.txt
echo 1000 | sudo tee $CA_ROOT/root-ca/serial
echo 1000 | sudo tee $CA_ROOT/root-ca/crlnumber
# Repeat for intermediate-ca/
```

LibOQS + OQS-Provider

What is LibOQS?

- Open source C library of PQ algorithms
- Backed by Open Quantum Safe (OQS) project
- Supported by Linux Foundation / PQCA
- Single API for all NIST-standardized algorithms

oqs-provider is an OpenSSL 3 Provider that wraps liboqs. PQ algorithms become natively available to the openssl CLI and any application linked against OpenSSL 3.

github.com/open-quantum-safe/liboqs

github.com/open-quantum-safe/oqs-provider

NIST-Standardized Algorithms

ML-KEM	FIPS 203	Key exchange
ML-DSA	FIPS 204	Signatures
SLH-DSA	FIPS 205	Alternate sig

Build Steps (A.1 & A.2)

```
# A.1 — build liboqs
git clone github.com/open-quantum-safe/liboqs
cmake -GNinja -DBUILD_SHARED_LIBS=ON
ninja && sudo ninja install && ldconfig

# A.2 — build oqs-provider
git clone github.com/open-quantum-safe/oqs-provider
cmake -GNinja -Dliboqs_DIR=...
ninja && sudo ninja install && ldconfig
```

CA Key Generation — LibOQS

A.3 — Configure OpenSSL Provider

```
# /opt/pq-ca/openssl-pq.cnf
[oqsprovider_sect]
module = /usr/local/lib/openssl-modules/oqsprovider.so
activate = 1

# Verify algorithms are available
export OPENSSL_CONF=/opt/pq-ca/openssl-pq.cnf
openssl list -signature-algorithms | grep ml-dsa
openssl list -kem-algorithms | grep ml-kem
```

A.5 — Intermediate CA (ML-DSA-65)

```
openssl genpkey -algorithm ml-dsa-65 \
-out intermediate-ca/private/int-ca.key.pem

openssl ca -extensions v3_intermediate_ca \
-days 3650 -notext -md sha256 \
-in csr/int-ca.csr.pem \
-out certs/int-ca.cert.pem
```

A.4 — Root CA (ML-DSA-87)

```
openssl genpkey -algorithm ml-dsa-87 \
-out root-ca/private/root-ca.key.pem
chmod 400 root-ca/private/root-ca.key.pem

openssl req -new -x509 -days 7300 \
-extensions v3_root_ca \
-key root-ca/private/root-ca.key.pem \
-out root-ca/certs/root-ca.cert.pem
```

A.6 — Server Certificate (ML-DSA-65)

```
openssl genpkey -algorithm ml-dsa-65 \
-out private/your-server.key.pem

openssl ca -extensions server_cert \
-days 825 -notext -md sha256 \
-in csr/your-server.csr.pem \
-out certs/your-server.cert.pem
```

SymCrypt + SymCrypt-OpenSSL Provider

What is SymCrypt?

- Microsoft's core cryptographic library
- Primary crypto engine in Windows since Win 8
- Open source · github.com/microsoft/SymCrypt
- FIPS 140-3 validated implementations

SymCrypt-OpenSSL wraps SymCrypt as an OpenSSL 3 Provider — a drop-in replacement for oqs-provider. Same openssl CLI commands and same certificate workflows.

Best for: government, regulated industries, FIPS 140-3 compliance requirements, organizations with Microsoft dependencies.

SymCrypt vs LibOQS

Feature	SymCrypt	LibOQS
FIPS 140-3 validated	✓	○
Algorithm breadth	○	✓
Prod hardening	✓	✓
Research tooling	○	✓
Windows interop	✓	○

Build Steps (B.1 & B.2)

```
# B.1 — build SymCrypt
git clone github.com/microsoft/SymCrypt
git submodule update --init # jitterentropy
python3 scripts/build.py cmake release --arch amd64
cp bin/release/.../libsymcrypt.so.* /usr/local/lib/
```

```
# B.2 — build SymCrypt-OpenSSL provider
git clone github.com/microsoft/SymCrypt-OpenSSL
cmake -GNinja -DSYMCRYPT_ROOT=../SymCrypt
ninja && sudo ninja install && ldconfig
```

CA Key Generation — SymCrypt

All genpkey / req / ca commands are identical to Option A — only OPENSSL_CONF changes to point at symcryptprovider.so

B.3 — Configure SymCrypt Provider

```
# /opt/pq-ca/openssl-symcrypt.cnf
[symcrypt_sect]
module = /usr/local/lib/openssl-modules/symcryptprovider.so
activate = 1

export OPENSSL_CONF=/opt/pq-ca/openssl-symcrypt.cnf
openssl list -signature-algorithms | grep ml-dsa
openssl list -kem-algorithms | grep ml-kem
```

B.5 — Intermediate CA (ML-DSA-65)

```
openssl genpkey -algorithm ml-dsa-65 \
-out intermediate-ca/private/int-ca.key.pem

openssl ca -extensions v3_intermediate_ca \
-days 3650 -notext -md sha256 \
-in csr/int-ca.csr.pem \
-out certs/int-ca.cert.pem
```

B.4 — Root CA (ML-DSA-87)

```
openssl genpkey -algorithm ml-dsa-87 \
-out root-ca/private/root-ca.key.pem
chmod 400 root-ca/private/root-ca.key.pem

openssl req -new -x509 -days 7300 \
-extensions v3_root_ca \
-key root-ca/private/root-ca.key.pem \
-out root-ca/certs/root-ca.cert.pem
```

B.6 — Server Certificate (ML-DSA-65)

```
openssl genpkey -algorithm ml-dsa-65 \
-out private/your-server.key.pem

openssl ca -extensions server_cert \
-days 825 -notext -md sha256 \
-in csr/your-server.csr.pem \
-out certs/your-server.cert.pem
```


Deploy to Apache

1. Copy Certificates

```
cp certs/your-server.cert.pem /etc/ssl/certs/  
cp private/your-server.key.pem /etc/ssl/private/  
cp certs/ca-chain.cert.pem /etc/ssl/certs/pq-ca-chain.cert.pem  
chmod 644 /etc/ssl/certs/your-server.cert.pem  
chmod 600 /etc/ssl/private/your-server.key.pem
```

2. Inject PQ Provider into Apache

```
# /etc/apache2/envvars.d/pq-openssl.conf  
# Option A: export OPENSSL_CONF=.../openssl-pq.cnf  
# Option B: export OPENSSL_CONF=.../openssl-symcrypt.cnf
```

3. Enable & Restart

```
a2enmod ssl headers  
a2ensite your-server-pq-ssl.conf  
apache2ctl configtest  
systemctl restart apache2
```

Verify PQ Handshake

```
openssl s_client -connect your-server:443 \  
-groups x25519_mlkem768 -tls1_3  
→ Look for: Server Temp Key: X25519_MLKEM768
```

Virtual Host — Key SSL Directives

```
SSLEngine on  
SSLCertificateFile  
    /etc/ssl/certs/your-server.cert.pem  
SSLCertificateKeyFile  
    /etc/ssl/private/your-server.key.pem  
SSLCertificateChainFile  
    /etc/ssl/certs/pq-ca-chain.cert.pem  
  
# TLS 1.3 only — ML-KEM requires TLS 1.3  
SSLProtocol -all +TLSv1.3  
  
# ML-KEM key exchange groups  
SSLOpenSSLConfCmd Groups \  
    "x25519_mlkem768:mlkem768:mlkem1024:x25519"  
  
SSLCipherSuite TLSv1.3 \  
    TLS_AES_256_GCM_SHA384  
  
SSLUseStapling on  
Header always set Strict-Transport-Security \  
    "max-age=63072000; includeSubDomains; preload"
```

Install the Root CA certificate in each client's trust store



Linux

```
sudo cp root-ca.cert.pem \  
/usr/local/share/ca-certificates/  
pq-root-ca.crt  
  
sudo update-ca-certificates
```

Copy root-ca.cert.pem to the system certificate store.

Distribute ONLY the Root CA cert — intermediate and server certs are served by Apache via SSLCertificateChainFile.



Windows

```
# PowerShell (Admin)  
Import-Certificate \  
-FilePath "pq-root-ca.crt" \  
-CertStoreLocation \  
"Cert:\LocalMachine\Root"
```

Imports the Root CA into the machine-wide Trusted Root store, so Edge, Chrome and .NET all trust the chain.

Run the shell as Administrator; a per-user import will not cover services.



macOS

```
sudo security add-trusted-cert \  
-d -r trustRoot \  
-k /Library/Keychains/  
System.keychain \  
pq-root-ca.crt
```

Adds the Root CA to the System keychain and marks it as a trusted root for SSL.

Safari and Chrome read this store directly; Firefox keeps its own and must be updated separately.

Certificate Revocation

1 Revoke a Certificate

```
# Revoke the certificate
openssl ca -config intermediate-ca.cnf \
  -revoke newcerts/1000.pem

# Regenerate CRL
openssl ca -config intermediate-ca.cnf \
  -gencrl \
  -out crl/intermediate-ca.crl.pem
```

2 Serve the CRL via Apache

```
mkdir -p /var/www/html/pki
cp crl/intermediate-ca.crl.pem \
  /var/www/html/pki/

# Add to intermediate-ca.cnf:
# crlDistributionPoints = URI:
# http://your-server/pki/
# intermediate-ca.crl.pem
```

Revocation Best Practices

CRL

Publish at URL embedded in `crlDistributionPoints`. Refresh every 30 days.

OCSP

Use OCSP for real-time status checking. Enable `SSLUseStapling` in Apache.

Backup

Back up `index.txt` and `serial` before any revocation operation.

Audit

Log all `openssl ca` operations. Review `CA index.txt` regularly.

Hybrid vs Pure Post-Quantum

Scenario	TLS Group	Rationale
Internet-facing, mixed clients	x25519_mlkem768	Classical fallback — compatible with non-PQ browsers
Internal / all PQ clients	mlkem1024	Maximum quantum resistance, no classical fallback
FIPS 140-3 compliance required	x25519_mlkem768	Use SymCrypt (Option B) for validated implementation
Research / prototyping	x25519_mlkem768 or mlkem768	LibOQS (Option A) — broadest algorithm selection
High-value Root CA key	N/A (cert signing)	ML-DSA-87 key, offline air-gapped machine

⚠ Pure ML-KEM-only TLS will fail on classical-only clients. Use x25519_mlkem768 hybrid for any internet-facing service until browser PQ support matures.

Security Hardening Checklist

✓ Root CA key is OFFLINE / air-gapped after Intermediate CA signing !

✓ TLS 1.3 is the ONLY enabled protocol (SSLProtocol -all +TLSv1.3) !

✓ CRL or OCSP endpoint published and reachable by clients

✓ Server cert ≤ 825 days · Intermediate ≤ 10 yr · Root ≤ 20 yr

✓ openssl ca audit logs reviewed regularly

✓ All private keys chmod 400, owned root:ssl-cert !

✓ Root CA key encrypted at rest (openssl genpkey -aes256) !

✓ HSTS preload header enabled on all virtual hosts

✓ liboqs / SymCrypt-OpenSSL pinned to a release tag (not main)

✓ Apache runs as www-data with no access to Root CA private key !

Summary

ML-DSA = Certificate Keys

CA root, intermediate, and server keys all use ML-DSA. Choose ml-dsa-87 for root CA, ml-dsa-65 for all others.

ML-KEM = TLS Key Exchange

Configured in Apache's SSLOpenSSLConfCmd Groups. Never appears as a certificate key algorithm.

Two Provider Options

LibOQS: broad algorithms, research-friendly. SymCrypt: FIPS 140-3 validated, Microsoft-backed, production-grade.

Hybrid First for Internet

Use x25519_mlkem768 until browser PQ support matures. Pure mlkem1024 for controlled internal networks only.

Air-Gap the Root CA

Generate root key offline. Never connect the Root CA to a network after signing the Intermediate CA.

Guide + Azure Automation

Guide: 40-Resources/pq-certificate-authority-ubuntu-2604.md. Azure scripts now include CA, Web, and Client VMs with 05-verify-tls.sh + 06-client-verify.sh.

Get the Lab

github.com/jringold/pqc-lab · Scripts for deploying PQC Certificate Authority Servers



What It Is

Automation scripts and step-by-step guides for standing up a post-quantum Certificate Authority test lab on Ubuntu 26.04 LTS and Windows Server nodes.



Azure Automation

Provisioning scripts build the CA, Web, and Client VMs end to end, with 05-verify-tls.sh and 06-client-verify.sh confirming the handshake.



PQC Algorithms

ML-DSA (FIPS 204) signs root, intermediate, and server certificates. ML-KEM (FIPS 203) handles TLS 1.3 key exchange — hybrid x25519_mlkem768 by default.



Provider Options

Build with LibOQS + OQS-provider for the broadest algorithm coverage, or Microsoft SymCrypt-OpenSSL for a FIPS 140-3 validated stack.



Quick Start

```
git clone https://github.com/jringold/pqc-lab
```

```
cd pqc-lab/azure · ./01-deploy.sh
```

```
./05-verify-tls.sh · ./06-client-verify.sh
```

The repo pairs the deployment scripts with the written guide (40-Resources/pq-certificate-authority-ubuntu-2604.md) so the whole CA — root, intermediate, and PQC-enabled web server — can be rebuilt from scratch and torn down on demand. Pre-release builds: test lab only, not production.

Four Ways to Build the PQC PKI Lab

The same ML-KEM / ML-DSA topology — on Azure or Hyper-V, with a Windows or Linux certificate authority

AZURE • WINDOWS

Azure Virtual Machines

Windows Server 2025
GA + KB5099536, or
vNext 29550+

- Azure CLI + PowerShell provisioning
- AD CS issuing ML-DSA certificates
- IIS terminating TLS 1.3 with ML-KEM

AZURE • LINUX

Automated PQ CA on Azure

Ubuntu 26.04 LTS with
LibOQS or SymCrypt
providers

- 11 scripts across 3 Azure VMs
- ~20 min LibOQS / ~30 min SymCrypt build
- 12 end-to-end server + client checks

HYPER-V • WINDOWS

Hyper-V, Windows CA

Windows Server 2025
GA + KB5099536, or
vNext 29550+

- PowerShell Direct into local VMs
- AD CS issuing ML-DSA certificates
- Same topology, browser verification

HYPER-V • LINUX

Hyper-V, Linux CA

Ubuntu 26.04 LTS with
LibOQS or SymCrypt
providers

- Bash + OpenSSL provider workflow
- Apache TLS 1.3 with ML-KEM
- Local-first mirror of the Azure build

All four paths issue ML-DSA (FIPS 204) certificates and negotiate ML-KEM (FIPS 203) key exchange — pick the one that matches your infrastructure.